

Adatbázis létrehozása

Adatbázis létrehozása

Az adatbázis létrehozása során elkészítettem az ingatlankezelő nevű MariaDB adatbázist a CREATE DATABASE SQL parancs segítségével. Ezt követően a USE utasítással kiválasztottam az adatbázist, hogy a további táblák és adatstruktúrák ebben a környezetben jöjjenek létre.

```
CREATE DATABASE ingatlankezelő;  
USE ingatlankezelő;
```

```
MariaDB [(none)]> CREATE DATABASE ingatlankezelő;  
Query OK, 1 row affected (0.003 sec)  
  
MariaDB [(none)]> USE ingatlankezelő;  
Database changed
```

1.Részlet a MariaDB -s adatbázis létrehozásáról.

Users tábla létrehozása

A users tábla létrehozása során megterveztem és elkészítettem a rendszer felhasználóinak adatainak tárolására szolgáló adatbázis táblát. A tábla kialakításakor meghatároztam a szükséges mezőket, például a felhasználó nevét, email címét, jelszavát, szerepkörét és telefonszámát, valamint beállítottam az elsődleges kulcsot és az egyedi email azonosítást is. A létrehozás után SQL lekérdezéssel ellenőriztem a tábla sikeres működését és szerkezetét a MariaDB rendszerben.

```
CREATE TABLE users (  
id INT AUTO_INCREMENT PRIMARY KEY,  
nev VARCHAR(100) NOT NULL,  
email VARCHAR(100) NOT NULL UNIQUE,  
jelszo VARCHAR(255) NOT NULL,  
szerepkor VARCHAR(50) NOT NULL,  
telefonszam VARCHAR(30),  
letrehozasdatum DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
MariaDB [ingatlankezelő]> CREATE TABLE users (  
-> id INT AUTO_INCREMENT PRIMARY KEY,  
-> nev VARCHAR(100) NOT NULL,  
-> email VARCHAR(100) NOT NULL UNIQUE,  
-> jelszo VARCHAR(255) NOT NULL,  
-> szerepkor VARCHAR(50) NOT NULL,  
-> telefonszam VARCHAR(30),  
-> letrehozasdatum DATETIME DEFAULT CURRENT_TIMESTAMP  
-> );  
Query OK, 0 rows affected (0.027 sec)  
  
MariaDB [ingatlankezelő]> SELECT*FROM users;  
Empty set (0.033 sec)
```

2.Részlet a users tábla létrehozásáról és az SQL kódjából.

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

Ingatlanok tábla létrehozása

Az ingatlanok tábla létrehozása során kialakítottam az ingatlanok adatainak tárolására szolgáló adatbázis szerkezetet. A tábla tartalmazza az ingatlan legfontosabb adatait, például a tulajdonos azonosítóját, az ingatlan címét, típusát, szobaszámát, havi bérleti díját és leírását. A fejlesztés során idegen kulcs kapcsolatot is létrehoztam a users tábla és az ingatlanok tábla között, amely biztosítja az ingatlanok és tulajdonosok közötti kapcsolat megfelelő kezelését az adatbázisban.

```
CREATE TABLE ingatlanok (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    tulajdonosid INT NOT NULL,  
    cim VARCHAR(255) NOT NULL,  
    tipus VARCHAR(100),  
    szobaszama INT,  
    haviar DECIMAL(10,2),  
    leiras TEXT,  
  
    CONSTRAINT fk_ingatlan_tulajdonos  
    FOREIGN KEY (tulajdonosid)  
    REFERENCES users(id)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

```
MariaDB [ingatlankezelelo]> CREATE TABLE ingatlanok (  
->     id INT AUTO_INCREMENT PRIMARY KEY,  
->     tulajdonosid INT NOT NULL,  
->     cim VARCHAR(255) NOT NULL,  
->     tipus VARCHAR(100),  
->     szobaszama INT,  
->     haviar DECIMAL(10,2),  
->     leiras TEXT,  
->  
->     CONSTRAINT fk_ingatlan_tulajdonos  
->     FOREIGN KEY (tulajdonosid)  
->     REFERENCES users(id)  
->     ON DELETE CASCADE  
->     ON UPDATE CASCADE  
-> );  
Query OK, 0 rows affected (0.026 sec)  
  
MariaDB [ingatlankezelelo]> SELECT * FROM ingatlanok;  
Empty set (0.001 sec)
```

3.Részlet az ingatlanok tábla létrehozásáról és az SQL kódjából.

Bérlők tábla létrehozása

A berlok tábla létrehozása során kialakítottam a rendszerben szereplő bérlők adatainak tárolására szolgáló adatbázis táblát. A tábla tartalmazza a bérlők legfontosabb adatait, például a nevüket, email címüket, telefonszámukat, személyi igazolvány számukat és lakcímüket. A fejlesztés során idegen kulcs kapcsolatot hoztam létre a users és berlok táblák között, amely biztosítja a felhasználói fiókok és a bérlői adatok összekapcsolását az adatbázisban.

```
CREATE TABLE berlok (  
id INT AUTO_INCREMENT PRIMARY KEY,  
userid INT NOT NULL UNIQUE,  
nev VARCHAR(100) NOT NULL,  
email VARCHAR(100),  
telefonszam VARCHAR(30),
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
szemelyiszam VARCHAR(50),  
lakcim VARCHAR(255),
```

```
CONSTRAINT fk_berlo_user  
FOREIGN KEY (userid)  
REFERENCES users(id)  
ON DELETE CASCADE  
ON UPDATE CASCADE  
);
```

```
MariaDB [ingatlankezelelo]> CREATE TABLE berlok (  
-> id INT AUTO_INCREMENT PRIMARY KEY,  
-> userid INT NOT NULL UNIQUE,  
-> nev VARCHAR(100) NOT NULL,  
-> email VARCHAR(100),  
-> telefonszam VARCHAR(30),  
-> szemelyiszam VARCHAR(50),  
-> lakcim VARCHAR(255),  
->  
-> CONSTRAINT fk_berlo_user  
-> FOREIGN KEY (userid)  
-> REFERENCES users(id)  
-> ON DELETE CASCADE  
-> ON UPDATE CASCADE  
-> );  
Query OK, 0 rows affected (0.024 sec)  
  
MariaDB [ingatlankezelelo]> SELECT * FROM berlok;  
Empty set (0.001 sec)
```

4.Részlet a bérlők tábla létrehozásáról és az SQL kódjából.

Szerződések tábla létrehozása

A szerzodesek tábla létrehozása során kialakítottam a bérleti szerződések adatainak tárolására szolgáló adatbázis szerkezetet. A tábla tartalmazza az ingatlan és a bérlő azonosítóját, valamint a szerződés kezdeti és végdátumát, havi díját és aktuális státuszát. A fejlesztés során idegen kulcs kapcsolatokat hoztam létre az ingatlanok és berlok táblákkal, amelyek biztosítják a szerződések megfelelő összekapcsolását az ingatlanokkal és a bérlőkkel az adatbázisban.

```
CREATE TABLE szerzodesek (  
id INT AUTO_INCREMENT PRIMARY KEY,  
ingatlanid INT NOT NULL,  
berloid INT NOT NULL,  
kezdetdatum DATE NOT NULL,  
vegdatum DATE NOT NULL,  
havidij DECIMAL(10,2),  
statusz VARCHAR(50),  
  
CONSTRAINT fk_szerzodes_ingatlan  
FOREIGN KEY (ingatlanid)  
REFERENCES ingatlanok(id)  
ON DELETE CASCADE  
ON UPDATE CASCADE,  
  
CONSTRAINT fk_szerzodes_berlo  
FOREIGN KEY (berloid)  
REFERENCES berlok(id)  
ON DELETE CASCADE
```

```
ON UPDATE CASCADE  
);
```

```
MariaDB [ingatlankezeselo]> CREATE TABLE szerzodesek (  
-> id INT AUTO_INCREMENT PRIMARY KEY,  
-> ingatlanid INT NOT NULL,  
-> berloid INT NOT NULL,  
-> kezdetdatum DATE NOT NULL,  
-> vegdatum DATE NOT NULL,  
-> havidijs DECIMAL(10,2),  
-> statusz VARCHAR(50),  
->  
-> CONSTRAINT fk_szerzodes_ingatlan  
-> FOREIGN KEY (ingatlanid)  
-> REFERENCES ingatlanok(id)  
-> ON DELETE CASCADE  
-> ON UPDATE CASCADE,  
->  
-> CONSTRAINT fk_szerzodes_berlo  
-> FOREIGN KEY (berloid)  
-> REFERENCES berlok(id)  
-> ON DELETE CASCADE  
-> ON UPDATE CASCADE  
-> );  
Query OK, 0 rows affected (0.027 sec)  
  
MariaDB [ingatlankezeselo]>  
MariaDB [ingatlankezeselo]> SELECT * FROM szerzodesek;  
Empty set (0.001 sec)
```

5.Részlet a szerzodesek tábla létrehozásáról és az SQL kódjából.

Fizetések tábla létrehozása

A fizetések tábla létrehozása során kialakítottam a bérleti szerződésekhez tartozó befizetések adatainak tárolására szolgáló adatbázis szerkezetet. A tábla tartalmazza a szerződés azonosítóját, a befizetett összeget, a fizetés dátumát, a fizetés státuszát, valamint a fizetési módot és a megjegyzéseket is. A fejlesztés során idegen kulcs kapcsolatot hoztam létre a szerzodesek és fizetesek táblák között, amely biztosítja a fizetések és a hozzájuk tartozó szerződések megfelelő összekapcsolását az adatbázisban.

```
CREATE TABLE fizetesek (  
id INT AUTO_INCREMENT PRIMARY KEY,  
szerzodesid INT NOT NULL,  
osszeg DECIMAL(10,2) NOT NULL,  
fizetesdatum DATE NOT NULL,  
statusz VARCHAR(50),  
fizetesmod VARCHAR(50),  
megjegyzes TEXT,  
  
CONSTRAINT fk_fizetes_szerzodes  
FOREIGN KEY (szerzodesid)  
REFERENCES szerzodesek(id)  
ON DELETE CASCADE  
ON UPDATE CASCADE  
);
```

```
MariaDB [ingatlankezo]> CREATE TABLE fizetesek (  
-> id INT AUTO_INCREMENT PRIMARY KEY,  
-> szerzodesid INT NOT NULL,  
-> osszeg DECIMAL(10,2) NOT NULL,  
-> fizetesdatum DATE NOT NULL,  
-> statusz VARCHAR(50),  
-> fizetesmod VARCHAR(50),  
-> megjegyzes TEXT,  
->  
-> CONSTRAINT fk_fizetes_szerzodes  
-> FOREIGN KEY (szerzodesid)  
-> REFERENCES szerzodesek(id)  
-> ON DELETE CASCADE  
-> ON UPDATE CASCADE  
-> );  
Query OK, 0 rows affected (0.021 sec)  
  
MariaDB [ingatlankezo]> ;  
ERROR: No query specified  
  
MariaDB [ingatlankezo]> SELECT * FROM fizetesek;  
Empty set (0.001 sec)
```

6.Részlet az fizetesek tábla létrehozásáról és az SQL kódjából.

Üzenetek tábla létrehozása

Az üzenetek tábla létrehozása során kialakítottam a felhasználók közötti üzenetek tárolására szolgáló adatbázis szerkezetet. A tábla tartalmazza az üzenet küldőjének azonosítóját, az üzenet tárgyát, szövegét és a küldés időpontját is. A fejlesztés során idegen kulcs kapcsolatot hoztam létre a berlok és üzenetek táblák között, amely biztosítja az üzenetek és a hozzájuk tartozó felhasználók megfelelő összekapcsolását az adatbázisban.

```
CREATE TABLE uzenetek (  
id INT AUTO_INCREMENT PRIMARY KEY,  
kuldoid INT NOT NULL,  
targy VARCHAR(255),  
uzenet TEXT NOT NULL,  
kuldesdatum DATETIME DEFAULT CURRENT_TIMESTAMP,  
  
CONSTRAINT fk_uzenet_kuldo  
FOREIGN KEY (kuldoid)  
REFERENCES berlok(id)  
ON DELETE CASCADE  
ON UPDATE CASCADE  
);
```

Szakmai Gyakorlat II. – Backend fejlesztési szakasz

Orosz Kristóf – EYZWG9

```
MariaDB [ingatlankezeslo]> CREATE TABLE uzenetek (  
-> id INT AUTO_INCREMENT PRIMARY KEY,  
-> kuldoid INT NOT NULL,  
-> targy VARCHAR(255),  
-> uzenet TEXT NOT NULL,  
-> kuldesdatum DATETIME DEFAULT CURRENT_TIMESTAMP  
->  
-> CONSTRAINT fk_uzenet_kuldo  
-> FOREIGN KEY (kuldoid)  
-> REFERENCES berlok(id)  
-> ON DELETE CASCADE  
-> ON UPDATE CASCADE  
-> );  
Query OK, 0 rows affected (0.022 sec)  
  
MariaDB [ingatlankezeslo]> SELECT * FROM uzenetek;  
Empty set (0.001 sec)
```

7.Részlet az uzenetek tábla létrehozásáról és az SQL kódjából.